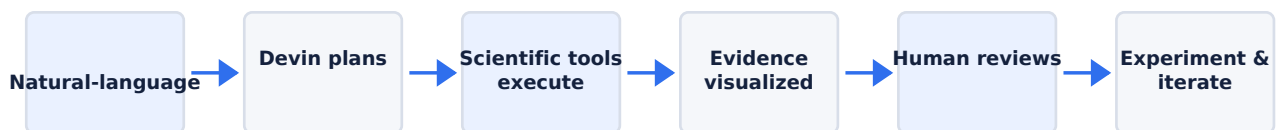


Devin-Powered Bioengineering Automation Platform

Hackathon build plan: prompt-driven scientific workflows for protein engineering, with a path to pharmaceutical discovery workflows.



Flagship demo

“Develop a PET-degrading enzyme that remains useful around 60 °C.” The product turns this objective into an inspectable computational investigation: homolog discovery, MSA, conservation, structure analysis, candidate generation, scoring, and a ranked experimental shortlist.

Product thesis

Devin should be visibly central to the product. It plans the investigation, operates scientific software in its sandbox, interprets outputs, repairs failed steps, responds to user constraints, and synthesizes evidence. The application owns persistent scientific state, permissions, visualization, and provenance.

Target hackathon outcome

- A working web interface where a scientist submits a biological engineering objective.
- A Devin session launches with a protein-engineering playbook and a preconfigured scientific environment.
- Devin autonomously executes several real bioinformatics tools and adapts when results or errors require it.
- The UI progressively renders workflow state, conservation/sequence evidence, structural evidence, and candidate recommendations.
- A follow-up user instruction changes the active Devin investigation rather than launching a hard-coded pipeline.
- Optional second round: ingest simple experimental measurements and have Devin propose the next computational round.

1. System architecture

Keep the application architecture thin around Devin. Do not implement a separate agent framework for the hackathon.



Responsibility split

Layer	Owns
Frontend	Project creation, prompt input, workflow progress, MSA/conservation/structure/variant visualizations, follow-up chat.
Backend	Project IDs, Devin session mapping, authentication, events, artifacts, structured results, guardrails.
Devin	Planning, tool selection, terminal execution, interpreting outputs, retrying failures, adapting to follow-ups, final synthesis.
Scientific tools	Actual sequence/structure calculations. Predictions remain predictions; wet-lab measurements are the ground truth.
Storage	PostgreSQL for metadata/state; object storage for FASTA, alignments, structures, JSON outputs, logs and reports.

Core rule

UI → backend → Devin. The browser should never hold Devin credentials or directly manage Devin sessions. The UI understands projects, stages, artifacts and candidates; the backend translates these into Devin sessions/messages.

Minimal services

- project-api - projects, runs, user objectives and constraints.
- devin-service - creates sessions, sends follow-up messages, reads status/results.
- artifact-service - indexes files/results produced by a run.
- event-stream - SSE/WebSocket updates from backend to UI.
- database - project/run/candidate/provenance state.

2. Repository and Devin environment

Create one repository that contains the product plus the thin scientific helpers Devin can use. Let Devin itself build and test as much of this repository as possible during the hackathon.

```
bio-platform/ frontend/ backend/ bio_tools/ sequence/ structure/ design/ visualization/ playbooks/
schemas/ tests/ AGENTS.md
```

Devin sandbox setup

Preconfigure a reusable Devin environment/snapshot with the repo and the lightweight scientific stack. For the hackathon, prefer tools that are fast, deterministic enough to demo, and easy to install.

Capability	Suggested implementation	Hackathon priority
Protein/sequence I/O	Python + Biopython	Must
Homolog search	MMseqs2	Must
Multiple sequence alignment	MAFFT	Must
Domain search	HMMER	Useful
Structure similarity	Foldseek	Must/Useful
Conservation	Python over MSA	Must
Sequence scoring	Protein LM if environment allows	Useful
Structure visualization data	PDB/mmCIF + annotations	Must
Heavy structure prediction	External/precomputed option	Optional
MD / large docking	External compute only	Skip for V1

Why thin wrappers?

For the first demo Devin can run installed CLIs directly. Add wrappers only where they make execution more reliable or outputs easier to parse. Example: `bioctl homolog-search`, `bioctl msa`, `bioctl conservation`, `bioctl candidate-sites`.

3. Put the scientific workflow in a Devin Playbook

The playbook defines the scientific operating procedure and decision rules. The individual tools perform calculations. Devin chooses and sequences them.

Protein engineering playbook - V1

Step	Instruction to Devin	Expected artifact
1	Validate the target, engineering objective and constraints.	objective.json
2	Collect relevant protein context and starting sequence/structure.	target.json
3	Search homologous proteins; assess whether the set is sufficiently diverse.	homologs.fasta + summary.json
4	Create an MSA and calculate per-position conservation.	alignment.fasta + conservation.json
5	Retrieve/search structural information; map important residues.	structure file + residue_annotations.json
6	Protect catalytic/highly constrained positions according to evidence.	protected_sites.json
7	Generate plausible candidate sites/variants using the available evidence.	variants.json
8	Run inexpensive scoring first; use heavier analysis only on a shortlist.	scores.json
9	Rank candidates across objectives and uncertainty.	ranking.json
10	Return a structured evidence-backed shortlist; never call predictions experimental results.	final_result.json

Agentic decision rules

- If homolog diversity is poor, broaden the search rather than blindly continuing.
- If a command fails, inspect the error/input, repair the issue, document the change and retry.
- Avoid expensive computation until cheaper filters have reduced the candidate set.
- If evidence conflicts, preserve the disagreement in the result instead of collapsing it into a false certainty.
- Respect user constraints added mid-run and re-evaluate affected downstream stages.

4. Connect Devin to the user interface

The UI should present a scientific investigation, not a terminal. Still expose enough agent activity to make Devin's autonomy obvious in the hackathon demo.

Backend endpoints

```
POST /projects POST /projects/{project_id}/runs GET /runs/{run_id} GET /runs/{run_id}/events GET /runs/{run_id}/results POST /runs/{run_id}/message GET /artifacts/{artifact_id}
```

Run lifecycle



Recommended UI views

View	What it shows
Overview	Research objective, constraints, Devin-generated plan, stage progress, current activity.
Protein family	Homolog counts, similarity/cluster view, candidate starting proteins.
Sequence	MSA viewer, conservation heatmap/sequence logo, protected positions.
Structure	Interactive 3D structure with catalytic, conserved and candidate residues.
Variants	Candidate funnel, ranked table, activity/stability-style multi-objective plots.
Evidence	For each candidate: calculated/predicted/inferred/experimental evidence and provenance.
Agent activity	Condensed timeline: command run, result observed, error fixed, decision made.
Experiments	Uploaded measurements, prediction-vs-measurement comparison, next-round recommendation.

Follow-up interaction

A user message such as “Exclude residues within 10 Å of the catalytic site” should be sent to the existing Devin session. Devin revisits the relevant artifacts and updates the candidate set; the backend persists the new result version and the UI refreshes.

5. Structured output and persistent state

Do not parse free-form Devin prose to drive the UI. Require a structured final result and separately persist run events/artifacts.

```
{ "objective": {...}, "plan": [...], "stage_results": [ {"stage": "homolog_search", "status": "complete", "artifact_ids": ["a_101"]} ], "candidate_sites": [...], "recommended_variants": [ { "variant_id": "v_008", "rank": 1, "evidence": {...}, "confidence": "medium", "experimental_status": "not_tested" } ], "limitations": [...] }
```

Minimal database model

Table	Important fields
projects	id, user/org, objective, target, constraints, created_at
runs	id, project_id, devin_session_id, playbook_version, status, started/completed
stages	run_id, stage, status, timestamps, summary
artifacts	id, run_id, type, path/URI, tool, tool_version, metadata
variants	id, project_id, parent, mutations, current_status
scores	variant_id, method, version, value, units, artifact_id
rankings	run_id, variant_id, rank, rationale/evidence references
experiments	project_id, round, variant_id, measurement type/value
events	run_id, type, payload, timestamp

Evidence taxonomy

KNOWN = database/literature fact; CALCULATED = direct software output; PREDICTED = model estimate; INFERRED = agent interpretation; EXPERIMENTAL = uploaded/measured result. Render these labels in the UI.

6. Visualizations to implement

For the hackathon, focus on visualizations that make the autonomous workflow understandable and scientifically inspectable.

Stage	Visualization	What the user learns
Planning	Workflow DAG / progress timeline	What Devin intends to do and what is running.
Homolog discovery	Similarity network or ranked homolog table	What protein family/search space was found.
MSA	Alignment viewer	Which positions align across homologs.
Conservation	Heatmap + sequence logo	Which positions are constrained or variable.
Structure	Mol* / NGL 3D viewer with residue overlays	Where catalytic/conserved/candidate residues sit in 3D.
Variant generation	Mutation-site map	Where Devin proposes engineering and why.
Screening	Funnel counts	How thousands of possibilities were reduced.
Ranking	Scatter/Pareto plot + table	Trade-offs between objectives; no fake single truth score.
Provenance	Evidence card/timeline	Exactly why a candidate survived.
Experiment round	Predicted vs measured plot	Whether the computational process is actually useful.

Best hackathon visual sequence



Keep the first version simple

If time is tight, implement: workflow progress + conservation heatmap + 3D protein viewer + ranked candidate table + agent activity timeline. Those five views tell the entire story.

7. Hackathon implementation sequence

Build vertically: get one complete Devin-driven path working before adding scientific depth.

Phase	Build	Exit criterion
0 - Prove Devin	Create a session from backend, run a simple repo command, retrieve structured output.	UI button can trigger a Devin run end-to-end.
1 - Sandbox	Install MMseqs2, MAFFT, Foldseek/Python helpers; smoke-test binaries.	Fresh Devin session can execute the scientific tools.
2 - Playbook	Implement protein-engineering V1 SOP and output schema.	Prompt causes Devin to create/execute the intended investigation.
3 - Artifacts	Standardize FASTA/MSA/conservation/structure/result files.	Backend can index and serve outputs.
4 - UI	Overview, activity timeline, conservation, structure, candidates.	A non-technical observer can follow the run visually.
5 - Steering	Send a follow-up constraint into the active session.	Candidate set visibly changes after user instruction.
6 - Error recovery	Demonstrate a real recoverable tool/input failure.	Devin diagnoses, fixes and retries without hard-coded recovery.
7 - Round 2	Upload small experimental result set and ask Devin for next candidates.	Full design-test-learn story works.

Suggested division of work

- Person A: Devin API integration + playbook + environment.
- Person B: bioinformatics wrappers/artifact schemas + test data.
- Person C: frontend scientific visualizations.
- Person D (if available): backend persistence/events + demo polish.
- Use Devin itself for repo scaffolding, wrapper implementation, tests, UI components, debugging and documentation wherever possible.

8. Five-minute demo script

The demo should emphasize autonomous scientific work rather than configuration screens.

Time	Demo action
0:00-0:30	Enter: "Develop a PET-degrading enzyme stable around 60 °C. Preserve catalytic function."
0:30-1:15	Show Devin-generated plan and live activity as it executes homolog search/MSA/conservation/structure tools.
1:15-2:00	Open conservation and 3D views. Show that evidence comes from actual artifacts, not generated prose.
2:00-2:40	Show variant funnel and candidate cards with evidence labels.
2:40-3:20	User says: "Exclude mutations within 10 Å of the catalytic site." Devin updates the analysis.
3:20-4:00	Show a tool failure/recovery trace or previously captured run: Devin detects malformed input, fixes and retries.
4:00-4:40	Upload a tiny Round-1 measurement set; ask Devin to analyze and propose Round 2.
4:40-5:00	Close on the architecture: Devin is both planner and scientific computer operator; the platform makes the work inspectable.

Pitch

"We turned Devin's software-engineering sandbox into an autonomous computational biology workspace. A scientist describes an engineering objective; Devin plans the investigation, operates scientific software, repairs broken workflows, interprets evidence, and iteratively proposes what to test next."

9. Scope, guardrails and what not to build

The hackathon wins by demonstrating agentic orchestration clearly, not by claiming to solve protein engineering end-to-end.

Must-have guardrails

- Never label a computational prediction as experimentally validated.
- Record the tool, version, parameters and artifact behind important results.
- Require human review before any physical experiment, synthesis/order, or expensive external compute.
- Keep Devin credentials and scientific service secrets server-side.
- Cap compute and expose approval boundaries for expensive jobs.
- Version the playbook and structured-output schema so results remain reproducible.

Do not spend hackathon time on

- Large-scale molecular dynamics infrastructure.
- Universal support for enzymes, antibodies, CRISPR, RNA and pharma at once.
- A custom agent loop that duplicates Devin.
- Perfect scientific ranking models.
- A full LIMS/ELN integration.
- Complex Kubernetes infrastructure unless already available.

After the hackathon

Version	Expansion
V2	Stability scoring + stronger candidate-generation models.
V3	Experimental-result ingestion + active-learning selection.
V4	External GPU jobs for structure prediction/docking/MD.
V5	Antibody engineering workflow.
V6	Small-molecule discovery workflow: target → compounds → docking/ADMET → shortlist.
V7	Enterprise integrations: LIMS/ELN, internal data, audit/compliance, RBAC.

10. Build checklist

Hackathon-ready definition of done

- Backend can create and track Devin sessions.
- Reusable Devin environment contains the required scientific tools.
- Protein-engineering playbook is versioned in the repo.
- Devin can run homolog search, MSA, conservation and at least one structure-oriented step.
- At least one real tool error can be autonomously recovered.
- Structured output validates against your schema.
- Artifacts are persisted and linked to the run.
- UI streams progress and agent activity.
- Conservation visualization works.
- 3D protein visualization works.
- Candidate shortlist includes evidence/provenance labels.
- Follow-up user constraint updates the existing run.
- Demo dataset is cached/reliable enough for live presentation.
- Optional: simple experimental-result upload triggers a next-round Devin analysis.

Recommended V1 stack

Next.js + FastAPI + PostgreSQL + S3-compatible object storage + SSE + Devin API + Devin environment containing Python/Biopython, MMseqs2, MAFFT and Foldseek. Add only the minimum wrappers needed to make tool invocation and artifacts predictable.

North-star metric

Time from biological question to an inspectable, evidence-backed candidate set. The product should reduce the human orchestration time between scientific intent and the next experimental decision.

11. CPU-first Devin Cloud execution plan

Constraint update: assume the Devin Cloud sandbox is CPU-only. The V1 workflow should therefore be designed so that the core scientific investigation runs locally on CPU, while GPU-heavy analysis is optional and delegated to external compute only when justified.

CPU-native scientific stack

Workflow task	CPU-friendly tool	Use in V1
Sequence / FASTA handling	Biopython	Yes
Homolog search + clustering	MMseqs2	Yes - core
Multiple sequence alignment	MAFFT	Yes - core
Domain/profile search	HMMER	Yes
Conservation / entropy	NumPy + Pandas / custom Python	Yes - core
Sequence logos	Logomaker	Yes
Structure retrieval	RCSB PDB + AlphaFold DB download	Yes - prefer retrieval over inference
Structure similarity	Foldseek	Yes - core
Structure parsing	Bio.PDB / Biotite	Yes
Secondary structure / exposure	DSSP	Yes
Pocket detection	fpocket	Useful
Mutation stability	FoldX	Useful after shortlist
Chemistry descriptors	RDKit	For pharma extension
CPU docking	AutoDock Vina	Optional / small shortlist
Trajectory analysis	MDAnalysis	Analysis only; avoid long local MD

Revised V1 PETase workflow

FASTA / target | v MMseqs2 homolog search | v MAFFT MSA | v Python conservation + sequence logo | v PDB / AlphaFold DB structure retrieval | v Foldseek + DSSP + Bio.PDB residue analysis | v Candidate mutation generation | v FoldX / lightweight CPU scoring | v Multi-objective ranking | v Evidence-backed shortlist

The important architectural change is that **structure prediction is no longer a required local stage**. First search experimental PDB structures and precomputed AlphaFold DB structures. Only escalate to an external GPU service when no suitable structure exists or higher-confidence re-prediction is necessary.

12. CPU vs external-compute policy

Keep inside Devin CPU sandbox	Escalate outside Devin when needed
MMseqs2 / BLAST-style search	AlphaFold/ColabFold inference
MAFFT	Large protein language-model inference
HMMER	Large ProteinMPNN generation batches
Conservation / entropy / statistics	Long molecular-dynamics simulations
Foldseek	Large neural docking campaigns
DSSP / Bio.PDB / Biotite	Large free-energy calculations
fpocket	High-volume GPU screening
FoldX on a shortlist	Any compute job exceeding configured cost/time limits
RDKit descriptors	

Agent decision policy

1. Prefer existing/precomputed biological data. 2. Run cheap CPU filters before expensive analysis. 3. Reduce the search space aggressively. 4. Use local CPU tools for the normal workflow. 5. If a GPU-only step would materially improve the result, request/submit an external job. 6. Continue with CPU evidence if the GPU step is optional. 7. Record whether every result was retrieved, calculated locally, predicted externally, or experimentally measured.

Updated hackathon story

“Devin operates a complete CPU-native computational biology workstation. It autonomously searches sequences, builds alignments, analyzes conservation and structures, proposes variants and ranks candidates locally. Only high-value deep-compute steps are escalated to external GPU services.”

Recommended Devin environment

Python biopython numpy pandas scipy scikit-learn logomaker biotite mdanalysis rdkit CLI mmseqs2 mafft hmmer foldseek dssp fpocket foldx autodock-vina (optional)

Revised hackathon priority

For the live demo, prove the entire path through **MMseqs2 -> MAFFT -> conservation -> structure retrieval/Foldseek -> residue analysis -> candidate generation -> CPU scoring -> ranking**. Treat GPU integration as a bonus architecture extension, not a dependency for the demo.